2048 个刻度

用一片 FPGA 完整替代 iC-TW28 细分器

项目介绍与技术原理说明 • 2026 年 11 月 • AMD 赛道 自主选题

01 一句话讲清这个项目

传感器转一圈，输出两路本应完美的正弦波，但实际是“歪”的；我用一片 FPGA 实时把它们算成精确角度，并把它做成一颗进口专用芯片（iC-TW28）的完整替代品。

02 要解决什么问题

电机、机器人关节、数控转台都需要知道自己“转了多少度”，这由编码器负责。光电编码器精度高，但依靠光路工作。现场有粉尘、油污或振动时，光路一旦受污染，就会出现细分误差突增甚至丢计数，这是产线上很典型的故障。

磁阻（SMR）传感器没有光路，非接触、抗污染，是替代方向；但它自身也有系统误差：

误差	物理来源	不校正时角度误差
直流偏置	电桥、放大器失调	13.99°
幅值失衡	两路增益差 1.25×	6.39°
正交偏差	不是正好差 90°（5°）	5.00°
三类叠加	——	20.98°

20° 意味着“让它转 90°，它报 70°”。所以必须校正，而校正正是 iC-TW28 这类进口专用芯片存在的理由。

03 项目做了什么

用一片通用 FPGA（Artix-7 XC7A35T）把 iC-TW28 的全部功能重新实现了一遍，而且更好。

信号在片内走一条固定 41.5 拍 ≈ 830 ns 的流水线，全程单一 50 MHz 时钟域（不例化 MMCM、不做跨时钟域，这是延迟可预测、无亚稳态的根本）：

ADC 同步采样 → 4 倍平均 → 六参数校正 → CORDIC 反正切 → PI 环路 → 多圈展开
→ ABZ / UVW / SPI / 串口 / 屏幕 / 千兆网

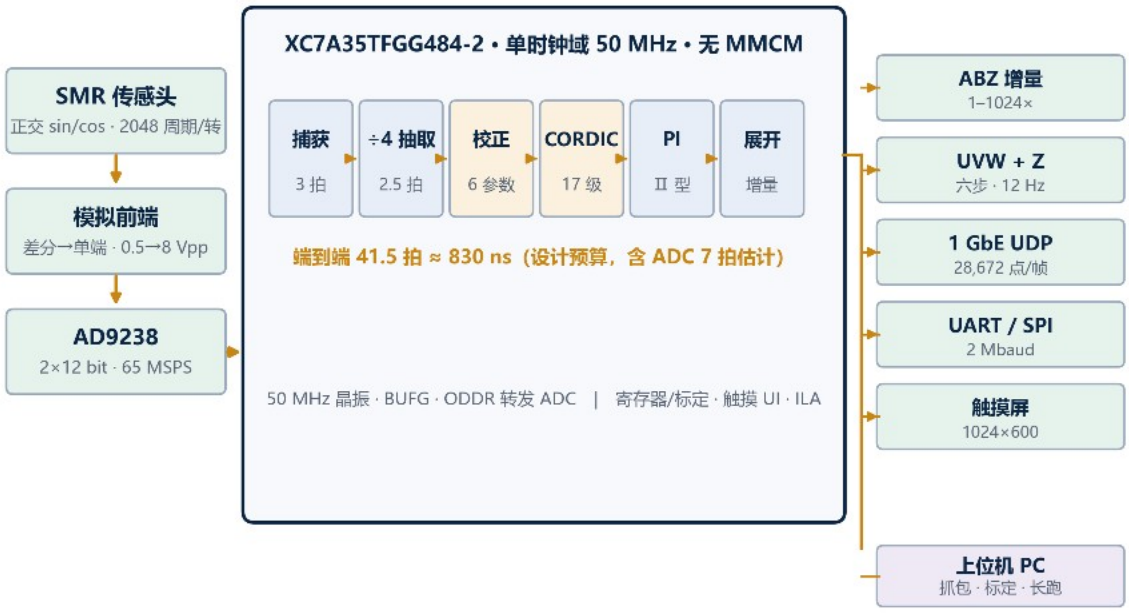


图 1　系统结构：左侧模拟前端与 ADC，中间单一 50 MHz 时钟域内的六级流水线，右侧六类输出

指标	本项目	iC-TW28
分辨率	16 bit / 65536 每周期（0.0055°）	10 bit（0.3516°）
校正参数	6 个，闭式算出，寄存器可读写	固件黑盒
接口	ABZ＋UVW＋SPI＋UART＋千兆网＋屏幕	SPI＋Encoder Link＋ABZ
延迟	≈830 ns	1.5 µs
算法	RTL 源码，可改可扩展	不可改

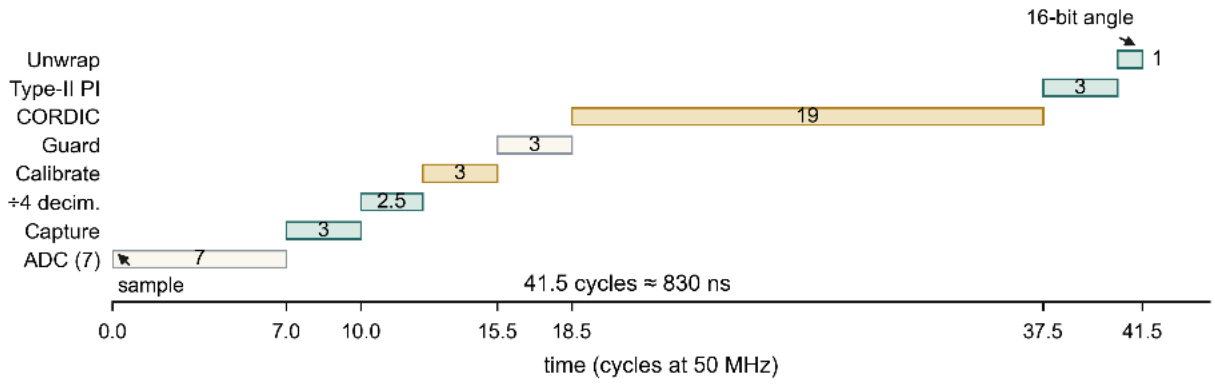


图 2　端到端延迟预算：仅 CORDIC 一级就占 19 拍，是整条链路的大头

04 技术原理：五步，每步都讲“为什么必须这么干”

① 同步采样

sin 和 cos 必须同一时刻采。差几十纳秒，在高速旋转时就等于引入随转速变化的相位误差。一片 AD9238 双通道共用采样保持，外部用两个 ODDR 以同相方式转发 50 MHz 时钟（早期反相驱动只剩 4 ns 余量，出过毛刺；改同相后余量回到 9.6 ns）。采完再做 4 点平均，等效分辨率由 12 位提高到约 13 位。

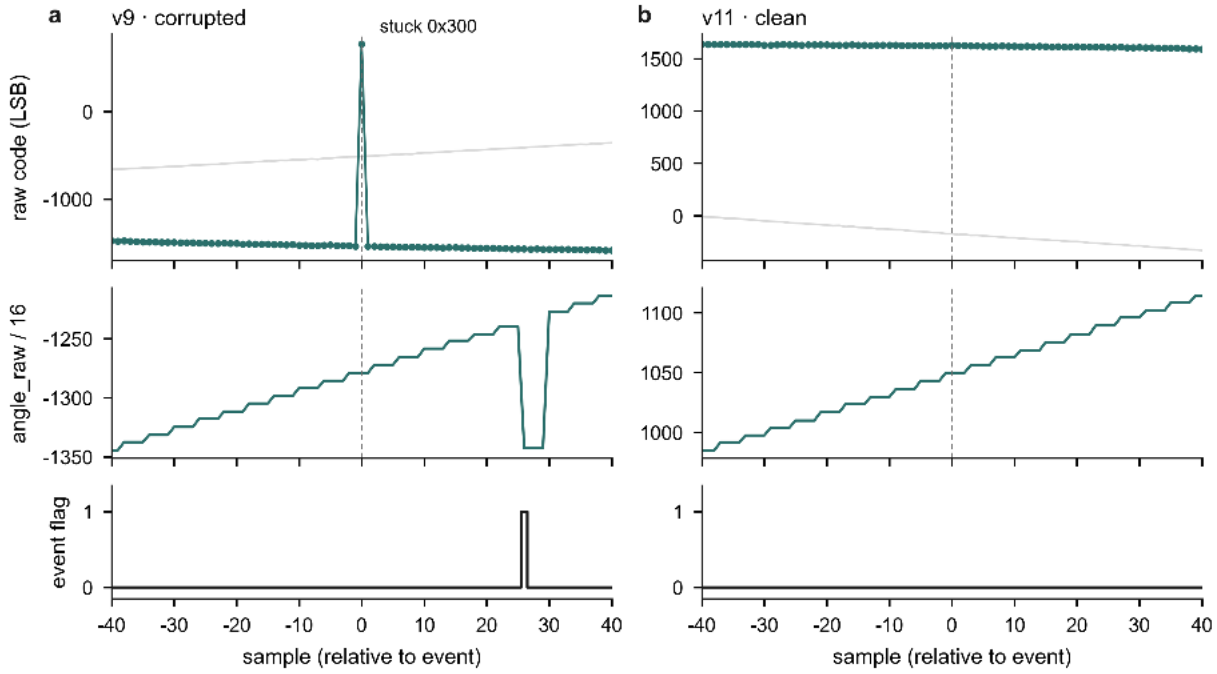


图 3　反相驱动（左）在采样点附近出现 0x300 卡死毛刺，角度塌陷 200 LSB；改同相驱动后（右）干净

② 六参数校正 —— 核心算法：把“歪椭圆”掰回“正圆”

把（sin, cos）画在平面上，理想的点落在一个单位圆上，极角就是角度。真实情况是落在一个歪掉的椭圆上：中心偏了、长短轴不等、轴还斜了。所以校正的本质是：找出 6 个数，把椭圆平移、缩放、剪切，变回圆。

关键在于不用迭代搜索，一次统计就能完成——转一圈后取均值得到偏置，取方差得到幅值，取协方差得到正交偏差，都是闭式公式。好处是无偏、对噪声不敏感，而且不需要知道输入频率。

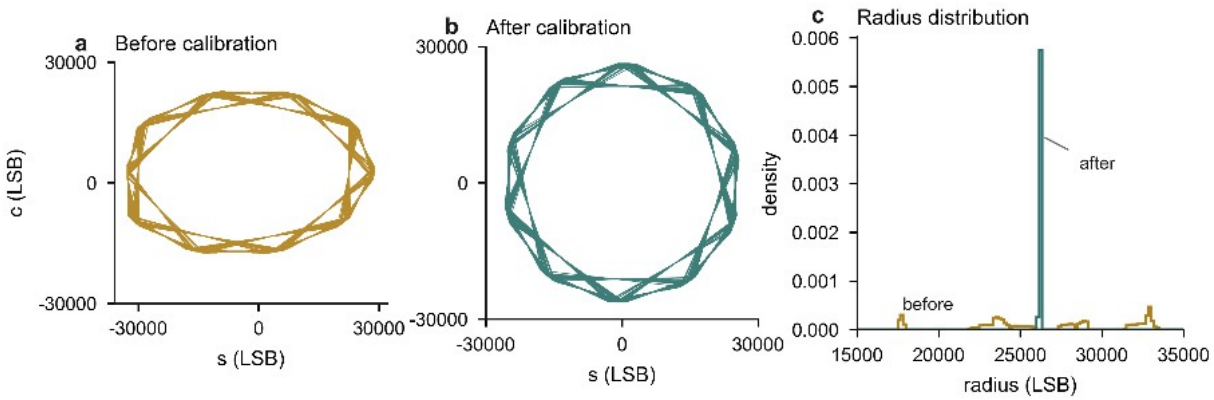


图 4　左：校正前的歪椭圆；中：校正后的圆；右：半径分布从散开变成一根尖峰

③ CORDIC —— FPGA 怎么算反正切

FPGA 不擅长除法、开方和三角函数。CORDIC 的思路很巧妙：把向量一次次往 x 轴上“掰”，每次只问“往哪边掰”，掰 17 次后向量基本躺在 x 轴上，把每次掰过的角度加起来就是答案。全程只用加法和移位（右移就是乘 2^{-i}），不需要乘法器，也不需要查找表。

④ PI 环路 —— 输出质量为何干净

CORDIC 输出会跟着 ADC 噪声抖，直接使用会让 ABZ 边沿抖动。加一个 II 型 PI 环路让输出角度去“追”CORDIC 角度：P 项追得快，I 项消稳态误差，积分器输出天然就是角速度。

这里有个真实的坑，也是最能证明“不是照搬”的地方：**照搬 iC-TW28 手册推荐的参数会发散振荡。**用 Jury 稳定判据推导后发现，若积分使用“上一拍的旧值”，稳定条件要求 Ki < Kp，而手册那组参数恰好违反；改成“使用本拍更新后的积分值”后，21 组参数全部稳定。

⑤ 多圈展开与输出

16 位只覆盖一个电周期。用跨零方向计数（不能用角度差分，零点抖动会多计或漏计，这是磁编码器最常见的故障），再加每转一个的 Z 脉冲对齐零点，然后按细分倍数生成 A/B 正交方波，按极对数生成 UVW 换相波。

05 为什么必须是 FPGA

	单片机	FPGA
数据率	50 MSPS×2×12 bit = 1.2 Gbit/s，收不进来	并行直接进
时序	中断、Cache、DMA 抖动	每 20 ns 一拍，零抖动
延迟	几十 μs 且不确定	830 ns 且固定 41.5 拍

伺服环对延迟抖动极敏感，这一点上 FPGA 是必需品而不是加分项。

06 做出来的是什么

不只是算法跑通，而是一整台仪器：转台真机 719–740 rpm 运行、7 寸触摸屏 5 页界面、千兆网一次回传 28,672 点波形 0.06 秒、PyQt 上位机在线标定、36 台 RTL 测试台全过、WNS +0.409 ns、DRC 0 错误。

实测：板上圆度 17.786% → 0.101%（约 176 倍），跟踪残差 0.009° RMS，仿真上限 1273 倍。

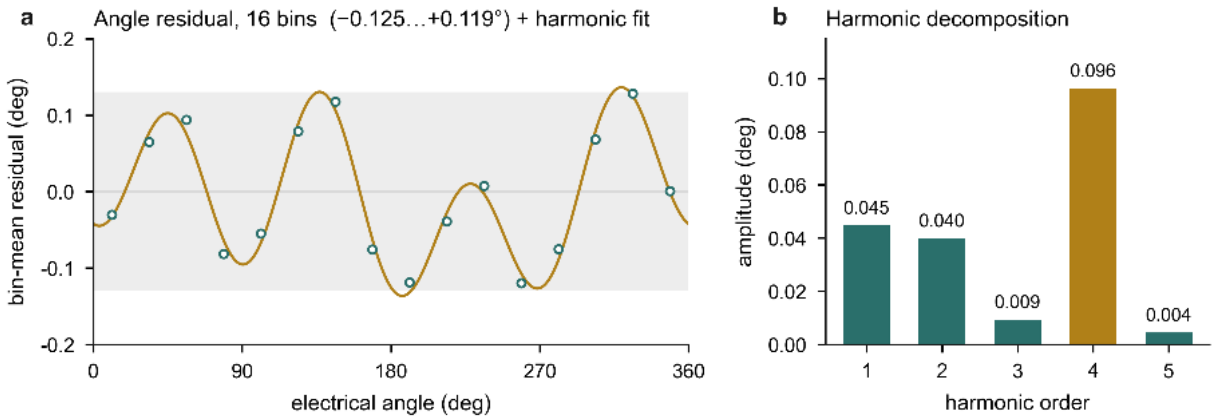


图 5 板上角度残差（16 bin，包络 −0.125…+0.119°）与谐波分解：4 次谐波 0.096° 占主导，剩余误差主要来自传感头与安装，而非校正算法

07 老师最可能问的三句话，你怎么答

“精度到底多少？”

分三个口径回答：仿真 20.98°→0.0165°、板上圆度 176 倍、跟踪残差 0.009° RMS。并主动补一句：分辨率 16 bit ≠ 精度，精度最终受传感器与安装偏心限制。

“为什么不直接用那颗进口芯片？”

它锁死了三件事：分辨率只有 10 位、算法是黑盒、接口固定。我们要的是更高分辨率和自定义接口，所以用 FPGA 把它“打开”。

“创新点在哪？”

不是“做了个编码器”，而是：把专用 SoC 的分辨率、算法和接口三把锁打开；自带仪表与数据通道；拥有可复核的证据链（54 台回归、DRC 0、259 行 SHA256）；参数化可移植。

08 一个必须记住的区别

仿真口径 vs 板端口径

“仿真 1273 倍”和“板上 176 倍”是两个口径：前者是模型在已知注入误差下的上限，后者包含真实噪声和安装误差。答辩时不能混着说——这是最容易被追问的地方。主动分开讲，比被问住强得多。

09 附：关键数字速查

指标	数值	口径
分辨率	16 bit / 电周期（0.0055° ）	设计规格
复合畸变（三类叠加）	20.9839° → 0.0165° （1273×）	定点模型，仿真
板端李萨如圆度	17.786% → 0.101%（176×）	板上实测
角度跟踪残差	0.009° RMS	板上实测
端到端延迟	41.5 拍 ≈ 830 ns	设计预算（含 ADC 7 拍估计）
资源占用（生产位流）	LUT 52.53% / BRAM 32/50 / DSP 17	v74 实现结果
时序	WNS +0.409 ns，0 失败端点	v74 实现结果
验证	36 台 RTL 测试台全过，DRC 0 错误	回归记录
以太网回传	28,672 点 / 帧，0.06 s	板上实测
真机转速	719 – 740 rpm，Z ≈ 12 Hz	转台实测

说明：本文件中的图表源文件位于 deliverables/figures/，位流、证据与回归记录位于 deliverables/。